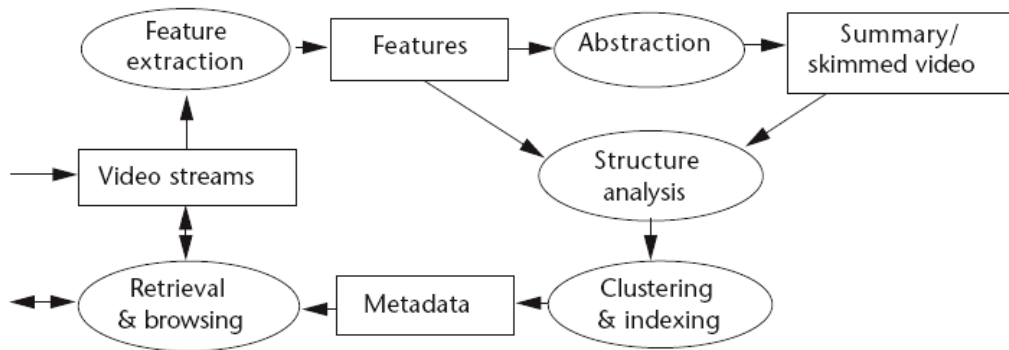


บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

2.1 พื้นฐานการค้นคืนวิดีโอ



รูปที่ 2.1 โค้ดโปรแกรมโปรเซสการวิเคราะห์และทำดัชนีวิดีโอ

จาก รูปที่ 2.1 นั้น แสดงถึงกระบวนการการวิเคราะห์, ทำดัชนี และการค้นคืนวิดีโอ [2] ซึ่งได้แบ่งออกเป็น การดึงลักษณะ (Feature extraction), การวิเคราะห์โครงสร้าง (Structure analysis), การสรุปวิดีโอ (Video abstraction) และการทำดัชนีและการค้นคืน (Indexing and Retrieval) โดยจะอธิบายตามหัวข้อดังต่อไปนี้

2.1.1 การดึงลักษณะ (Feature extraction)

ในโปรเซสนี้ ถือว่าเป็นโปรเซสที่สำคัญที่สุดในเรื่องของการค้นคืนวิดีโอ ประสิทธิภาพของการดัชนีนั้นจะขึ้นอยู่กับตัวแทนหลักที่บ่งบอกถึงเนื้อหา นั่น ๆ ซึ่งในขั้นตอนนี้เป็นโปรเซสที่ยากที่สุด โดยจะพยายามดึงลักษณะของวิดีโอออกมา ไม่ว่าจะเป็น สี, รายละเอียด, รูปร่าง, โครงสร้าง, รูปแบบ หรือ การเคลื่อนที่ของกล้อง ซึ่งนำมาสร้างเป็น แนวความคิดเชิงความหมาย (Semantic concept) ความคิดเชิงความหมายนี้เอง อาจมีการนำเรื่องของเสียงมาวิเคราะห์ร่วมด้วย เพื่อสร้างความเชื่อถือให้กับความคิดเชิงความหมายได้มากยิ่งขึ้น

ในการพิจารณาข้อมูลวิดีโอซึ่งประกอบด้วย วัตถุ (object) เหตุการณ์ (event) และฉาก (scene) ความสัมพันธ์ระหว่างองค์ประกอบเหล่านี้ เรียกว่า แนวความคิดเชิงความหมาย (semantic concept) ซึ่งภายในประกอบด้วยลักษณะระดับสูง (high level feature) เช่น เหตุการณ์และวัตถุ และ ลักษณะระดับต่ำ (low level feature) เช่น สี, รูปร่าง, พื้นผิว, ขอบ หรือ เวกเตอร์การเคลื่อนที่ ซึ่งช่องว่างระหว่างทั้งสองระดับนี้ เรียกว่า ช่องว่างทางความหมาย (semantic gap) ซึ่งการจำแนกที่ได้นั้นควรมีช่องว่างนี้น้อยที่สุด เพราะลักษณะระดับต่ำนั้นสามารถหาได้โดยใช้เทคนิคการประมวลผลภาพ แต่ในขณะที่ลักษณะระดับสูงเป็นความหมายตามความเข้าใจมนุษย์

โดยเหตุผลดังกล่าวได้มีผู้เสนอวิจัยเกี่ยวกับเวกเตอร์ลักษณะเชิงความหมาย (Semantic feature) โดยนำเวกเตอร์ลักษณะระดับล่างมาสร้างแบบจำลองตรวจหาลักษณะเชิงความหมาย หรือ สร้างตัวจำแนก เพื่อนำมาอธิบายวัตถุ เหตุการณ์ หรือฉากที่เกิดขึ้นในวิดีโอเพื่อใช้ในการทำดัชนีสำหรับการค้นคืนวิดีโอ และการให้คำประกอบวิดีโอ (Video Annotation)

2.1.2 การวิเคราะห์โครงสร้าง (Structure analysis)

ขั้นตอนต่อมาคือการวิเคราะห์โครงสร้างของวิดีโอ หลังจากการดึงลักษณะออกมาแล้ว ซึ่งโปรเซสนี้จะเป็นการแยกส่วนวิดีโอทางเวลา (Temporal video extraction) ซึ่งจะเป็นการบ่งบอกถึงขอบเชิงเวลา และ บอกถึงความหมายในแต่ละช่วงที่ได้แบ่งแยกไว้ โดยจะมีการแบ่งไว้ในลักษณะช็อต (shot), เฟรมสำคัญ (keyframe) และ ฉากวิดีโอ (scene) ซึ่งจะถูกรวบรวมและจัดประเภทคล้ายกับการสร้าง storyboard ในการแบ่งกลุ่มนั้น (ตามภาพที่ 1) จะเริ่มจากกลุ่มของฉาก และ แบ่งย่อยลงไปถึงช็อต และ ในแต่ละช็อตจะหาเฟรมตัวแทน หรือที่เรียกว่า เฟรมสำคัญ เพื่อจะนำไปสร้างดัชนีในการค้นหาวิดีโอต่อไป

การหารอยแบ่งแต่ละช็อตนั้นมีวิธีการมากมายหลายอย่าง โดยวิธีที่ธรรมดาที่สุดคือ การเปรียบเทียบลักษณะของสีในเฟรมที่ติดกัน, รายละเอียด, ขอบของวัตถุ, และ ความแตกต่างระหว่างเฟรมที่มีแนวโน้มว่าจะเป็นรอยต่อของช็อต โดยรอยต่อของช็อตนี้เอง เรียกว่า “hard cut” ตามลักษณะทางกายภาพของรอยต่อนี้เอง ได้แบ่งประเภทออกเป็น การเปลี่ยนแปลงของช็อตแบบทันทีทันใด และ การเปลี่ยนแปลงของช็อตแบบค่อยเป็นค่อยไป เช่น fade หรือ dissolve ในประเภทหลังนับว่าการหา hard cut นั้นจะยากกว่าวิธีแรก แต่ในหลักการนั้นยังมีวิธีที่คล้ายกันแทนที่จะวิเคราะห์จากเฟรมที่อยู่ติดกัน เป็น วิเคราะห์เฟรมที่ไม่ได้อยู่ติดกันนั่นเอง

การแบ่งแต่ละฉากนั้นออกเป็นช็อตวิดีโอ นั้น มีความสำคัญในการที่จัดกลุ่มแบ่งประเภทของกลุ่มของฉากและช็อตวิดีโอ โดยการนำช็อตวิดีโอที่มีความคล้ายคลึงกันมารวมกัน เพื่อที่จะนำมาวิเคราะห์ต่อในระดับที่สูงขึ้น ซึ่งในการวิเคราะห์ระดับสูงนี้ได้แบ่งออกเป็น 2 ประเภทซึ่งทั้งสองนั้นเป็นวิธีแบบอัตโนมัติโดยอาศัยหลักการจดจำ (Recognition) ได้แก่ อาศัยกฎการผลิตรายชื่อ และ อาศัยลำดับความสำคัญจากผู้ใช้งาน ในวิธีทั้งสองนี้ยังคงมีข้อจำกัดเนื่องจากทั้งสองหลักการนี้นั้น ยังคงแสดงความหมายในระดับเฉพาะบุคคล (subjective semantic) ซึ่งอาจได้มาจากผู้พัฒนาระบบ และนับว่าการวิเคราะห์เหล่านั้นไม่เป็นสากล แต่นับว่าเป็นเรื่องที่โชคดีอย่างยิ่ง ช็อตวิดีโอที่นับว่าเป็นตัวเลือกที่ดีตัวหนึ่งในการดัชนีสำหรับการค้นหา ซึ่งอาศัยหลักการเปรียบเทียบกับเฟรมสำคัญกับคำค้นหาจากผู้ใช้งาน

2.1.3 การสรุปวิดีโอ (Video abstraction)

การสรุปวิดีโอ เป็นโปรเซสในการสร้างตัวแทนของวิดีโอและต้องมีความยาวที่น้อยกว่าวิดีโอต้นฉบับ ซึ่งโปรเซสนี้คล้ายกับการดึงเอาคำสำคัญ (key word) หรือ บทสรุปออกมา ดังนั้น การดึงเอาเฟรมสำคัญ “Key frame” หรือ high light ที่เข้าถึงในแต่ละช็อต, ฉาก หรือ เรื่องราว จากวิดีโอต้นฉบับออกมาเป็นบทสรุปของช็อตนั้นๆ โปรเซสนี้ นับว่ามีความสำคัญมากที่สุดในการสร้างการสรุปข้อมูลในวิดีโอ ไม่เพียงจะได้ตัวแทนของวิดีโอเท่านั้น แต่ยังสามารถสรุปการค้นหาวงจรของวิดีโอออกมาด้วยเช่นกัน ซึ่งสิ่งเหล่านี้เองบางครั้งสามารถสร้างตารางตัวเลือกในรูปแบบภาพแทนเนื้อหาเพื่อสำหรับการค้นหาได้ (browsing)

เฟรมสำคัญเป็นเซตของเฟรมในวิดีโอที่เลือกมาเป็นตัวแทนของช็อตวิดีโอที่มีความสำคัญมากที่สุดภายในช็อตนั้นๆ ซึ่งเฟรมสำคัญนี้เป็นภาพนิ่งที่ถูกดึงออกมาจากวิดีโอต้นฉบับ ประสิทธิภาพของเฟรมสำคัญนี้ขึ้นอยู่กับวิธีที่เลือกเฟรมสำคัญนี้ออกมาว่าสามารถบ่งบอกถึงเนื้อหาภายในแต่ละช็อตได้มากน้อยเพียงไร ดังนั้นการค้นหาเฟรมสำคัญได้มีเทคนิคออกมาต่างๆ มากมาย ในที่นี้จะยกตัวอย่างมา 4 เทคนิคดังนี้

1. แนวความคิดแบบรอยต่อของช็อต (shot boundary based approach)

หลังจากวิดีโอผ่านขั้นตอนของการแยกวิดีโอทางเวลาแล้วจะได้หน่วยย่อยที่เรียกว่าช็อต วิธีการอย่างง่ายในการดึงเฟรมสำคัญออกมานั้น คือ นำเฟรมแรกของแต่ละช็อตวิดีโอมาเป็นเฟรมสำคัญ แต่วิธีการนี้เป็นวิธีที่ง่ายแต่เฟรมสำคัญในแต่ละช็อตจะถูกจำกัดเพียงแค่เฟรมเดียวเท่านั้น

2. แนวความคิดแบบเนื้อหาด้วยการมองเห็น (Visual Content Based approach)

Zhang และคณะ [3] ได้นำเสนอหลายกฎเกณฑ์การมองเห็นเพื่อที่จะดึงข้อมูลเฟรมสำคัญ คือ แทนที่จะใช้เพียงเฟรมเดียวของแต่ละช็อตวิดีโอ ก็เลือกหลายเฟรมสำคัญจากช็อตวิดีโอ เรียกว่า เซตของเฟรม (Set of Key frame) โดยมีกฎดังนี้

- กฎเกณฑ์พื้นฐานของช็อต (Shot based criteria) คือการดึงเฟรมสำคัญในระดับช็อต โดยในแต่ละเฟรมแรกถือว่าสำคัญที่สุด ส่วนเฟรมสำคัญที่เหลือขึ้นอยู่กับวิเคราะห้ในขั้นตอนต่อมา
- กฎเกณฑ์พื้นฐานของลักษณะสี (Color feature based criteria) คือ หลังจากได้เฟรมแรกของช็อตวิดีโอที่เป็นเฟรมสำคัญแล้ว จะเปรียบเทียบความเหมือนกันระหว่างเฟรมแรกกับเฟรมที่เหลือ โดยอาศัยหลักวิธีฮิสโตแกรมสี หรือ โมเมนต์สี (color moment)
- กฎเกณฑ์พื้นฐานของการเคลื่อนที่ เป็นการปรับปรุงโดยใช้ข้อมูลของการเคลื่อนที่รวม (global motion) ที่ได้จากการดำเนินการของกล้อง (camera operation) และการเคลื่อนที่ของวัตถุที่ขนาดใหญ่ในวิดีโอ เนื่องจากการดำเนินการของกล้องนั้น ใช้เพื่อหาเหตุการณ์ หรือ ต้องการเน้นวัตถุอย่างชัดเจน ซึ่งจะตรวจลักษณะการเคลื่อนที่คือ การส่าย (panning) และการซูม (zooming) ดังนั้นจำนวนของเฟรมสำคัญจะขึ้นอยู่กับอัตราการเปลี่ยนแปลงในการมองเห็นเนื้อหาภาพ นั่นคือเฟรมสำคัญที่ได้จากการส่ายจะต้องมีการซ้อนทับกันน้อยที่สุด โดยทั่วไปแล้วเฟรมสำคัญควรมีเนื้อหาในภาพน้อยกว่า 30%

3. แนวความคิดแบบการวิเคราะห์การเคลื่อนที่ (Motion analysis based approach)

Wolf และคณะ [4] ได้นำเสนอการดึงเฟรมสำคัญโดยพิจารณาการเคลื่อนที่ กล่าวคือใช้การวิเคราะห์การไหลเชิงแสง (Optical Flow Analysis) เพื่อเป็นวัดค่าการเคลื่อนที่ที่เกิดขึ้นในช็อตวิดีโอ และเลือกเฟรมสำคัญจากช่วงที่มีค่าการเคลื่อนที่น้อยที่สุดซึ่งช่วงนี้ เรียกว่า ค่าที่น้อยที่สุดของช่วง (Local minima)

4. แนวความคิดแบบการจัดกลุ่ม (Clustering based approach)

Zhuang และคณะ ได้เสนอวิธีการจัดกลุ่ม ที่ใช้ในการรู้จำรูปแบบ เพื่อที่จัดกลุ่มเฟรมในข้อดีวิดีโอเพื่อหาเฟรมสำคัญ หลังจากการจัดกลุ่มให้แต่ละเฟรมแล้วนั้นต่อไปก็คือการเลือกหาเฟรมสำคัญ โดยที่กลุ่มที่มีขนาดใหญ่ที่สุดเพียงพอที่จะเป็นกลุ่มสำคัญ (key cluster) มากกว่าค่าขนาดเล็กๆ จะมีเฟรมสำคัญเพียงเฟรมเดียวที่ได้จากกลุ่มที่ใหญ่ที่สุด ถ้ามีขนาดเล็กกว่าค่าเฉลี่ย เฟรมสำคัญหาได้จากเฟรมที่มีค่าใกล้เคียงกับค่าศูนย์กลางของแต่ละกลุ่ม

2.1.4 การทำดัชนีและการค้นคืน (Indexing & Retrieval)

หลังจากผ่านกระบวนการดึงลักษณะ, การวิเคราะห์วิดีโอ และ การสรุปวิดีโอ ข้อมูลที่ได้มานั้นจะถูกจัดเก็บไว้ในฐานข้อมูลเฉพาะด้าน (metadata) คือ ได้ทั้งข้อมูลสรุปจากการวิเคราะห์วิดีโอ และ ตัววิดีโอเอง จากข้อมูลที่ได้มานั้นสามารถนำมาสร้างเป็นดัชนีและตารางของเนื้อหาไว้สำหรับการค้นหาและการค้นคืน ซึ่งในการสร้างดัชนีนี้เอง ต้องมีการผ่านกระบวนการตัดและแยกแยะลำดับหรือข้อดีเป็นกลุ่ม หรือโครงสร้างดัชนี ในระบบฐานข้อมูลนั้นด้วย เพื่อต้องการเครื่องมือสำหรับการดึง, ค้นหา ในวิดีโอที่มีขนาดใหญ่ ๆ ได้

2.2 ภาพรวม OpenCV Library

ไลบรารีนี้เน้นไปที่การทำงานกับการแสดงผลแบบเรียลไทม์(real time) ตัวอย่างของงานได้แก่ การตอบสนองระหว่างคนกับคอมพิวเตอร์ (Human-Computer Interaction) การวิเคราะห์วัตถุ การแบ่งและวิเคราะห์ การวิเคราะห์หน้า การวิเคราะห์การเคลื่อนไหว การติดตามความเคลื่อนไหว อัตราการเคลื่อนไหว การทำความเข้าใจเกี่ยวกับการเคลื่อนไหว การวิเคราะห์โครงสร้างจากความเคลื่อนไหว (SFM) และหุ่นยนต์ไร้สาย

2.2.1 Intel ® Image Processing Library

- 2.2.1.1 การสร้างภาพและการเข้าถึงเพื่อแก้ไข(Header เดียวใช้สำหรับ 2 ไลบรารี)
- 2.2.1.2 การทำงานด้านคณิตศาสตร์และตรรกศาสตร์กับภาพ
- 2.2.1.3 Images filtering
- 2.2.1.4 การแปลงภาพแบบ Linear
- 2.2.1.5 การทำงานเกี่ยวกับโครงสร้างของภาพ
- 2.2.1.6 การแปลง Color Space
- 2.2.1.7 Image histogram และ Thresholding

- 2.2.1.8 การเปลี่ยนแปลงในแบบเรขาคณิต ซึ่งได้แก่การหมุน หมุน เบ้ สะท้อนภาพ บิด เปลี่ยนมุมมอง เลียนแบบ
- 2.2.1.9 การทำงานกับภาพ ณ ช่วงเวลาหนึ่ง

2.2.2 ตัวอย่างApplication

- 2.2.2.1 การสอนเกี่ยวกับการใช้กลุ่มเครื่องมือการแบ่งแยกกล้องของ Matlab
- 2.2.2.2 Automatic camera calibration filter
- 2.2.2.3 การติดตามสี การติดตามใบหน้า
- 2.2.2.4 Condensation filter fracker
- 2.2.2.5 การวิเคราะห์หน้าโดยใช้ embedded HMMs
- 2.2.2.6 Kalman filter tracker
- 2.2.2.7 Lucas-Kanade optical flow in an image pyramid

2.2.3 โปรแกรมช่วยเหลือที่มีคนช่วยสร้างขึ้น(Windows)

- 2.2.3.1 ตัวอย่างการหาโปรแกรมติดตั้งกล้องของ Direct Show ด้วย CAMSHIFT
- 2.2.3.2 Matrox Meteor Direct Show capture filter

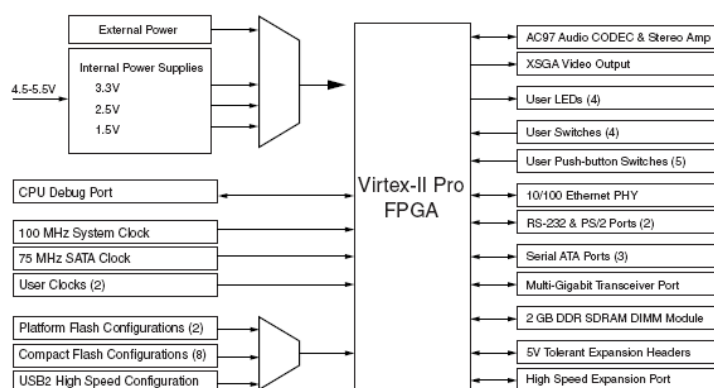
2.2.4 โปรแกรมช่วยเหลือที่มีคนช่วยสร้างขึ้น (Linux)

- 2.2.4.1 BMP to IPL file reader/writer
- 2.2.4.2 การหาค่าเฉลี่ยและค่าสูงสุดต่ำสุดของชุดข้อมูลที่อยู่นับสิบ

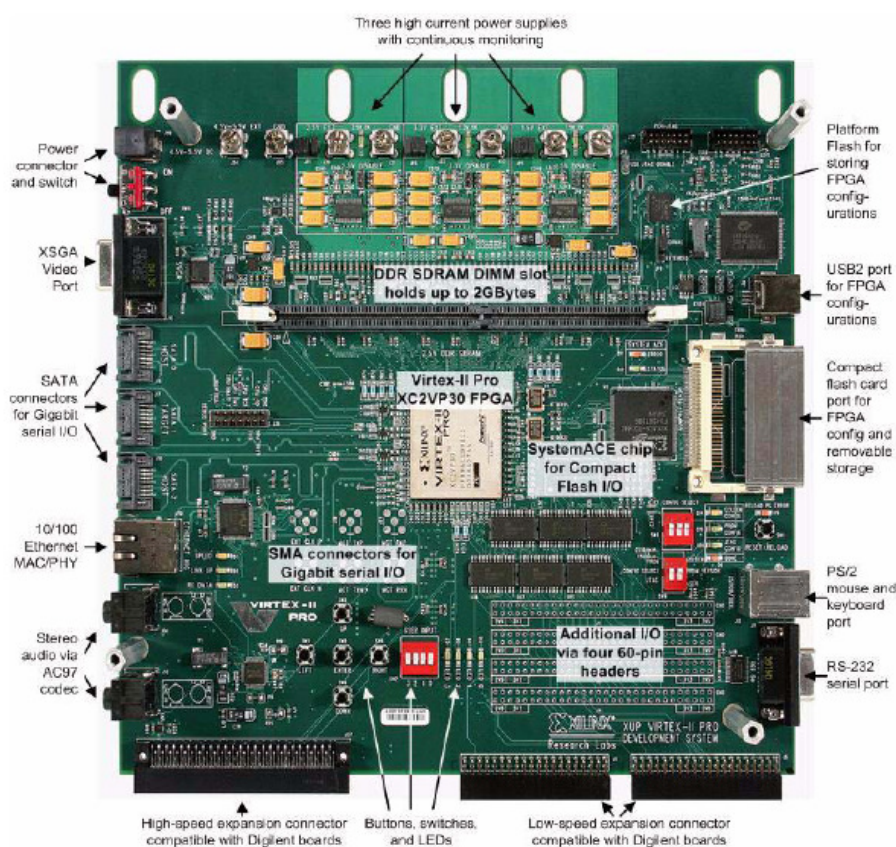
การศึกษาการใช้งานไลบรารี OpenCV นั้นศึกษาหลักเบื้องต้น ได้จาก [5] จะอธิบายหลักการ ทำงานอย่างคร่าว พร้อมทั้งตัวอย่าง code

2.3 Xilinx Virtex-II pro FPGA

บอร์ดที่ใช้ในการพัฒนาทางด้าน Hardware คือ บอร์ดรุ่น Virtex-II pro FPGA ซึ่งนับว่าเป็น FPGA ที่มีประสิทธิภาพสูงในการประมวลผล อีกทั้งยังมี Interface ที่ครบสมบูรณ์แบบ ส่งผลให้สามารถ ทำงานได้อย่างซับซ้อน โดยเฉพาะด้าน Image & Video processing



รูปที่ 2.2 ภาพไออะแกรมของ Virtex-II pro FPGA



รูปที่ 2.3 ภาพบอร์ด Xilinx Virtex-II Pro FPGA

ส่วนสัญญาณวิดีโอ นั้น ในโครงงานนี้จะรับเข้ามาจากสัญญาณที่วีธีรรมดาแบบ SDTV โดยสามารถเสียบได้ทั้งสาย Component Video, S-Video และสาย Composite Video เข้ากับตัว VDEC1

(Video Decoder Board) ซึ่งตัว VDEC1 นี้เป็นตัวเสริมแยกต่างหากโดยใช้ชิป ADV7183B เป็นตัวถอดรหัสสัญญาณวิดีโอที่เข้ามาจึงสามารถรองรับสัญญาณได้ทั้ง NTSC, PAL และ SECAM (ในที่นี้เราจะใช้ PAL เป็นหลัก) ออกมาเป็น ITU-R BT.656 Digital Format จากนั้นจึงติดต่อเข้ากับตัว system board ผ่านพอร์ตการใช้งานแบบ Hirose FX2 Data Connector โดยมีค่า Output ออกไปยัง system board ด้วยระบบสีแบบ YCrCb 4:2:2

เมื่อต่อ VDEC1 เข้าสู่ตัว system board แล้ว จำเป็นต้องทำการโปรแกรมชิป ADV7183B เพื่อรองรับการทำงานเสียก่อนจึงจะสามารถนำสัญญาณเข้าไปยัง system board ได้ ไม่เช่นนั้นจะทำให้เกิดการชนกันของพอร์ตข้อมูลที่เข้าสู่ชิป FPGA ในตัว system board ได้

2.4 การใช้งาน FPGA ควบคู่กับ Image Processing

จากการศึกษาข้อดี ข้อเสียของหลักการทำ Image Processing ของไฟล์ประเภทวิดีโอแล้ว ไฟล์วิดีโอธรรมดา จะมี frame rate อยู่ที่ 25 fps (ในกรณีการเข้าสัญญาณทีวีแบบ PAL) ซึ่งหมายความว่า ในหนึ่งวินาที จะมีภาพที่ระบบต้องนำมาใช้คำนวณทั้งหมด 25 ภาพ (ซึ่งแล้วแต่อัลกอริทึมที่นำมาใช้) และในกระบวนการทำ การเปรียบเทียบภาพแต่ละภาพ มักจะสิ้นเปลือง process และเวลาในการคำนวณ หลักการต่างๆ ยิ่งถ้าต้องการเพิ่มความแม่นยำของการเปรียบเทียบให้ได้ข้อมูลที่ถูกต้องชัดเจนยิ่งขึ้น จะต้องนำไปผ่านกระบวนการต่างๆ เพิ่มขึ้นอีกในแต่ละเฟรม ซึ่งจะทำให้เกิดภาระอย่างหนักขึ้นกับตัวเครื่องคอมพิวเตอร์ที่ใช้ทำงาน ดังนั้นทางผู้ทำโครงการจึงได้ทำการศึกษาระบบประมวลผลภายนอก แยกเฉพาะออกไป ทางนักศึกษาได้ตัดสินใจเลือกใช้ FPGA เพราะเป็นที่นิยมของตลาด และมีข้อมูลทางการวิจัยและผลิตภัณฑ์ที่ออกมาสู่ตลาดออกมามากมาย โดยทางบริษัท Xilinx ซึ่งเป็นบริษัทที่ผลิตได้แสดงผลลัพธ์ของการทำงานประมวลผลทาง Image Processing ไว้ว่าได้ผลที่เร็วกว่าถึง 6 เท่า

ในการออกแบบบน FPGA มีวิธีต่างๆ ที่น่าสนใจหลายวิธี โดยวิธีต่างๆ มีข้อดีข้อเสีย ที่ได้เรียงไว้ในตารางข้างล่างนี้ ทางนักศึกษาได้เลือกใช้วิธี System Generator หลังจากนั้นค่อยไปปรับแต่ง Code ภาษ VHDL เอาในภายหลัง

ตารางที่ 2.1 การเปรียบเทียบข้อดี ข้อเสีย แนวทางในการพัฒนา FPGA

วิธีการออกแบบ	ข้อดี	ข้อเสีย
Full VHDL/Verilog (RTL Code)	1) มีความคล่องตัวสูง 2) สามารถควบคุมการออกแบบงานได้อย่างสมบูรณ์ 3) สามารถแก้ไขและทำความเข้าใจใน Code ได้ง่ายกว่า	1) อาจต้องใช้เวลาในการออกแบบนาน 2) ไม่สามารถควบคุมการทำงานของเครื่องมือสังเคราะห์วงจรได้อย่างตลอดเวลา 3) ต้องการความเข้าใจในภาษาที่ใช้ออกแบบอยู่พอสมควร 4) ต้องทำงานร่วมกับเครื่องมือสังเคราะห์วงจรเพื่อให้ได้มาซึ่งการออกแบบวงจรที่สมบูรณ์
Full VHDL/Verilog (Instantiating Primitives)	1) เข้าถึงโครงสร้างแกนหลักของระบบได้ดีที่สุด 2) สามารถเจาะลึกรายละเอียดได้มากที่สุด 3) มีประสิทธิภาพสูงที่สุด	1) ไม่คล่องตัวเท่ากับแบบ RTL 2) ต้องการความเข้าใจในระบบสถาปัตยกรรมของ FPGA เป็นอย่างดี 3) ใช้เวลาในการออกแบบนาน
CORE Generator	1) สามารถเข้าถึงและใช้งานฟังก์ชันที่มีอยู่แล้วอย่างรวดเร็ว 2) ไม่จำเป็นต้องสร้างรูปแบบวงจรขึ้นมาใหม่ถ้าตรงกับความต้องการแล้ว 3) IP ถูกออกแบบอย่างสมบูรณ์กับบางวงจรแล้ว	1) IP อาจจะไม่ทำงานตรงตามอย่างที่ต้องการเสมอไป 2) ต้องอาศัยความเข้าใจของสัญญาณและพารามิเตอร์ที่จะนำเข้ามาสู่ระบบการออกแบบด้วย 3) ทำงานกับระบบปิดที่ทราบข้อมูลเพียงเล็กน้อย

ตารางที่ 2.1 (ต่อ) การเปรียบเทียบข้อดี ข้อเสีย แนวทางในการพัฒนา FPGA

วิธีการออกแบบ	ข้อดี	ข้อเสีย
System Generator	1) สามารถสร้างผลิตภัณฑ์ได้มากจากระบบการออกแบบขั้นสูง 2) มีความสามารถในการทดสอบการออกแบบในระดับขั้นวงจร 3) เหมาะสำหรับผู้เริ่มใช้งาน FPGA รายใหม่ๆ 4) มีความสามารถในการออกแบบ testbench ขั้นสูงที่พร้อมเพรียง 5) HDL Testbench, test vector และข้อมูลต่างๆ ถูกเขียนขึ้นให้โดยอัตโนมัติ 6) ฮาร์ดแวร์ที่อยู่ในการทดสอบโดยการเชื่อมต่อจำนวนรอบเพิ่มประสิทธิภาพของตัวผลิตภัณฑ์ และสามารถตรวจสอบความผิดปกติของวงจรได้อย่างรวดเร็วและเรียบง่าย	1) อาจจะได้ไม่ผลลัพธ์ที่สมบูรณ์แบบมากนักในมุมมองของการใช้งานจริง 2) ผู้ใช้งานจำเป็นต้องศึกษาการใช้งาน Simulink ไปด้วย 3) ไม่เหมาะสำหรับระบบที่มีสัญญาณนาฬิกาหลายตัวทำงานร่วมกัน 4) ไม่สนับสนุนระบบบัสสองทาง

2.4 เริ่มต้นการใช้งาน Simulink

Simulink เป็นซอฟต์แวร์ที่สามารถทำให้ผู้ใช้งานจำลองการทำงานของระบบ และสามารถวิเคราะห์ผลลัพธ์ที่อาจมีการเปลี่ยนแปลงไปได้ตามช่วงเวลาของการทำงาน Simulink สามารถนำมาใช้ตรวจสอบระบบการทำงานที่เปลี่ยนแปลงไปตามเวลาได้หลากหลายรูปแบบ ตัวอย่างเช่น วงจรไฟฟ้า ระบบขับเคลื่อนแรงกระตุ้น ระบบเบรก รวมไปถึง ระบบไฟฟ้า เครื่องกล และระบบความร้อน

การจำลองระบบการทำงานที่เปลี่ยนแปลงไปตามเวลา มีการประมวลผลสองระดับใน Simulink อย่างแรก ผู้ใช้ สร้าง บล็อกไดอะแกรม ใช้ Simulink model editor ซึ่งเป็นตัวคำนวณ

ความสัมพันธ์ทางคณิตศาสตร์ของค่าที่รับเข้ามาของระบบ สถานะของระบบ และผลลัพธ์ของระบบ โดยผู้ใช้สามารถกำหนดระยะเวลาที่จะให้ระบบจำลองการทำงานได้

2.4.1 Creating Models

Simulink นำเสนอ editor ที่อนุญาตให้ผู้ใช้สร้างและต่อบล็อกที่เลือกมาจากไลบรารี ในไลบรารีบล็อกได้อัตโนมัติ Simulink มีไลบรารีของบล็อกที่จำลองระบบขั้นพื้นฐานเพื่อการใช้งานไว้มากมาย ผู้ใช้ Simulink สามารถสร้างบล็อกของตัวเองขึ้นมาและนำมาจำลองในระบบได้เองด้วย

2.4.2 Custom Blocks

Simulink อนุญาตให้ผู้ใช้สร้างไลบรารีของบล็อกที่สร้างขึ้นเองโดยผู้ใช้ สามารถนำมาใช้จำลองในระบบได้ ผู้ใช้สามารถสร้างบล็อกได้จากทั้งในรูปแบบกราฟิกหรือโดยการเขียนโปรแกรมลงไปได้ การสร้างบล็อกโดยการเขียนโปรแกรมนั้น สามารถสร้างได้จาก M-file หรือ MEX-file ที่เก็บคำสั่งการทำงานของบล็อกเอาไว้ โดยเราจะเรียกไฟล์นี้ว่า S-function เราสามารถนำ S-function เข้ามาใช้งานร่วมกับ Simulink ในบล็อกที่เราสร้างขึ้นได้ทันที โดยสามารถเพิ่มค่าตัวแปรจาก parameter dialog เข้าไปใช้ใน S-function ของเรา โดยเขียนฟังก์ชันขึ้นมารองรับเพิ่มเติมตามจำนวนค่าตัวแปรได้

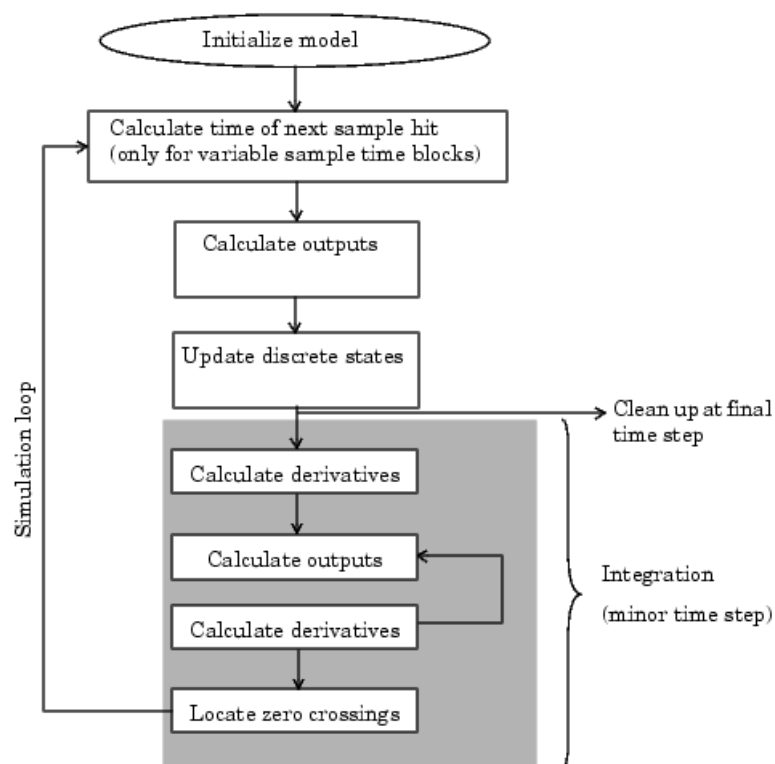
2.4.3 Systems and Subsystems

บล็อกไดอะแกรมของ Simulink สามารถรองรับงานเป็นชั้นๆได้ ในแต่ละชั้นจะเรียกว่า “ระบบย่อย” (subsystem) ระบบย่อยเป็นส่วนหนึ่งของระบบทั้งหมด ซึ่งมีไว้เพื่อการจัดการรูปแบบของบล็อกไดอะแกรมให้เป็นหมวดหมู่มากขึ้น โดย Simulink แยกความแตกต่างของระบบย่อยไว้สองชนิดคือ เสมือนจริง (virtual) กับ ไม่เสมือนจริง (nonvirtual) สิ่งที่แตกต่างกันนั้นคือใน ระบบย่อยแบบ nonvirtual นั้นสามารถที่จะเข้าไปควบคุมว่าจะให้ รับหรือส่งผลลัพธ์ออกไปได้เมื่อไหร่ ซึ่งจากการศึกษาบบย่อยและการใช้งานแล้ว ทางผู้จัดทำได้เลือกใช้ระบบย่อยในรูปแบบไม่เสมือนจริง เพราะสามารถควบคุมผลลัพธ์ที่จะออกมาตามเวลาที่ต้องการได้

2.4.4 Simulation Stages

การทำงานของ Simulink model กระทำเป็นทีละขั้นตอน เริ่มจากการตั้งค่า ในกระบวนการนี้ Simulink มองบล็อกออกมาเป็น model กำหนดขนาดความกว้าง ชนิดของข้อมูล และจังหวะของเวลา ตรวจสอบค่าพารามิเตอร์ของแต่ละบล็อก จัดเตรียมลำดับการคำนวณของทุกบล็อก และเตรียมขนาดของหน่วยความจำเพื่อรองรับการทำงานทั้งหมดไว้ จากนั้น Simulink จะเข้าสู่การจำลองระบบงาน โดยการทำงานเสร็จในแต่ละรอบจะเรียกว่าลำดับขั้นการจำลอง ในแต่ละลำดับขั้นการจำลอง Simulink จะ

คำนวณผลลัพธ์ที่จะออกมาของแต่ละบล็อก ที่ได้จากการตั้งค่าไว้แล้วในตอนต้น โดยในแต่ละบล็อก Simulink จะเรียกใช้ function ที่คำนวณสถานะของบล็อก ค่าที่เปลี่ยนแปลง และผลลัพธ์ในจังหวะเวลานั้น และวนทำเช่นนี้ต่อไปจนจบการจำลอง ภาพข้างล่างแสดงขั้นตอนการทำงานของการทำงานของการจำลองระบบใน Simulink



รูปที่ 2.4 แสดงการทำงานของ Simulink ในแต่ละรอบจังหวะเวลา

2.4.5 Writing Level-2 M-File S-Functions

Level-2 M-file S-function API ทำให้ผู้ใช้งานสามารถใช้ภาษา M ของ MATLAB ในการสร้างบล็อกของตัวเองขึ้นมาได้โดยไม่จำกัดจำนวนอินพุตและเอาต์พุตและยังรองรับสัญญาณใดๆที่เกิดขึ้นในระบบ Simulink ได้ทั้งหมด รวมทั้งค่าเมตริกซ์ของทุกชนิดข้อมูล

Level-2 M-file S-function คือ M-file ที่เก็บค่าคำสั่งและลักษณะการทำงานของบล็อกที่ผู้ใช้สร้างขึ้นเพื่อนำมาใช้ใน Simulink model โดยผู้ใช้สามารถปรับเปลี่ยนโค้ดได้ตามใจชอบ เพื่อรองรับการทำงานจากการจำลองได้ ซึ่งในการเขียนแบบ M-file นี้ ผู้ใช้สามารถนำฟังก์ชันต่างๆใน MATLAB มาใช้งานได้ในพื้นที่เหมือนกับการเขียน M-file ทั่วไป นอกจากนี้ S-function ยังสามารถเขียนได้ในภาษาอื่นๆอีก เช่น ภาษา C, Pascal, Ada, Fortran โดยสามารถแปลงมาเป็นไฟล์ที่ใช้งานใน MATLAB

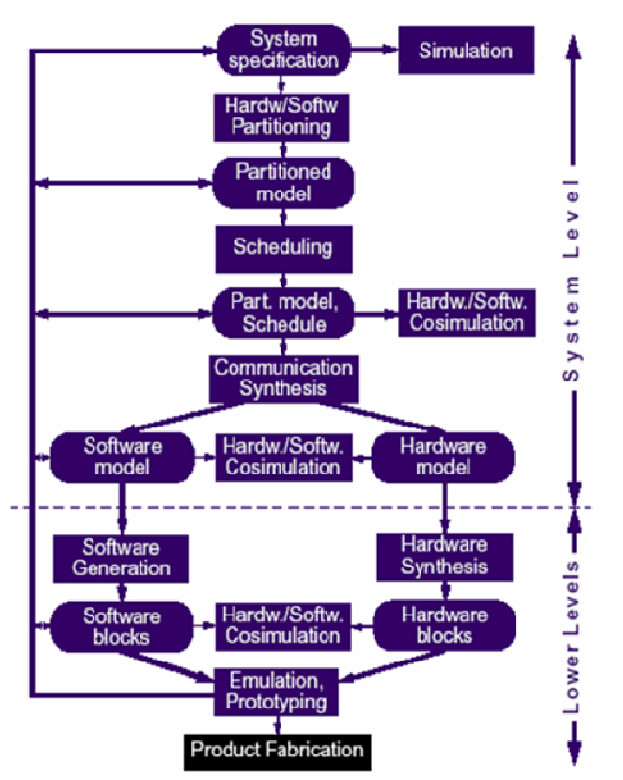
ได้ ในโครงการนี้จะมุ่งไปที่การเขียน Level-2 M-file S-function ในแบบ M-file เพราะความง่ายของการใช้งานและประยุกต์ใช้ร่วมกับระบบ Simulink

2.5 การออกแบบร่วมกันระหว่าง ฮาร์ดแวร์ และ ซอฟต์แวร์ (Hardware–Software Codesign)

นิยามของ Codesign ที่ให้โดย Franke และ Purvis ในบทความในปี 1991 ว่า “เป็นการออกแบบระบบที่มีทั้งฮาร์ดแวร์ และซอฟต์แวร์ พร้อม ๆ กันตั้งแต่จุดเริ่มต้นของการออกแบบ เพื่อความยืดหยุ่นในการออกแบบ และประสิทธิภาพของการจัดการฟังก์ชันการทำงานในระบบอย่างเหมาะสม” Wayne Wolf ให้นิยามเพิ่มเติมว่า “ระบบจะต้องออกแบบร่วมกันทั้งในส่วนของฮาร์ดแวร์ และซอฟต์แวร์ เพื่อเป็นการประกันว่าสามารถทำงานได้ในตอนท้ายที่สุด ซึ่งมีประสิทธิภาพ เชื่อถือได้ และราคาตามที่ต้องการ ”

ดังนั้นการออกแบบร่วมกันระหว่างฮาร์ดแวร์-ซอฟต์แวร์เป็นสร้างระบบที่มีความสมดุลกันระหว่างประสิทธิภาพ (System performance) ที่ได้มาจากฮาร์ดแวร์ และราคา (System cost) ที่ได้มาจากซอฟต์แวร์ หลักการของCodesign คือการหลีกเลี่ยงการแยกกันของการออกแบบฮาร์ดแวร์ และการออกแบบซอฟต์แวร์ โดยได้ผลลัพธ์เป็นระบบที่มีประสิทธิภาพ

Hardware software codesign ในภาพที่ 2.5 นั้น ผู้ออกแบบเริ่มต้นออกแบบจากภาษาชั้นสูง (High-level design language) ในการกำหนดความสามารถของระบบ (System specification) หลังจากนั้นก็จะพยายามออกแบบทั้งในส่วนของ ฮาร์ดแวร์และซอฟต์แวร์ไปพร้อม ๆ กัน โดย codesign tools เช่น *Partitioner* ที่แบ่งส่วนงานให้ไปทำที่ฮาร์ดแวร์หรือซอฟต์แวร์ *Scheduler* ที่เรียงลำดับการทำงานในส่วนต่าง ๆ ของระบบทั้งที่อยู่ในฮาร์ดแวร์ และซอฟต์แวร์ โดยมีการกระทำที่เรียกว่า *Cosimulation* เพื่อทดสอบการทำงานร่วมกันตลอดเวลา



รูปที่ 2.5 แสดงการพัฒนาแบบ Reconfigurable computing system

2.5.1 ประเภทของการออกแบบร่วมกันระหว่างฮาร์ดแวร์-ซอฟต์แวร์

การประยุกต์ใช้งานในลักษณะของการออกแบบร่วมกันระหว่างฮาร์ดแวร์-ซอฟต์แวร์มีได้หลากหลาย โดยมีรากฐานมาจากระบบฝังตัว หรือ Embedded system ซึ่งเป็นระบบที่ประกอบด้วยไมโครโปรเซสเซอร์ และอุปกรณ์ฮาร์ดแวร์ เพื่อช่วยเพิ่มประสิทธิภาพของระบบ ยิ่งถ้าเราสามารถที่จะทำฮาร์ดแวร์ในระบบมีความยืดหยุ่นได้โดยใช้ FPGA ก็จะทำให้มีความน่าสนใจในการใช้งานเพิ่มขึ้นอย่างมากทีเดียว แบ่งลักษณะของระบบ Codesign ให้เป็นสองลักษณะใหญ่ๆ ดังนี้คือ

- Embedded system: ระบบจะประกอบด้วย CPUs และทรัพยากรในระบบต่าง ๆ เช่น ASIC, FPGA, หน่วยความจำ (Memory), การอินเตอร์เฟสกับผู้ใช้ (Human interface), พอร์ตสำหรับการวิเคราะห์ (Diagnostic port), ตัวแปลงสัญญาณ ADC/DAC, อุปกรณ์ตรวจจับ (Sensor) และอุปกรณ์ทางกล (Actuator) ระบบนี้มักจะออกแบบขึ้นมาเพื่อจุดประสงค์เฉพาะอย่าง เช่น บอร์ดควบคุมในเครื่องซักผ้าระบบเบรกในรถยนต์ หรือระบบบังคับการบินในเครื่องบิน เป็นต้น ถ้าจะแยกย่อยลงไปอีกเรา

สามารถแบ่งเป็นระบบที่มีการตอบสนอง (Interact) สอดคล้องสถานะในการใช้งานจริง โดยมีตัวอุปกรณ์ตรวจจับและอุปกรณ์ทางกล ในการใช้งานกับส่วนต่าง ๆ ของระบบให้มีการตอบสนองเป็นลักษณะของ Real-time reactive ภายใต้เงื่อนไขที่กำหนดให้ทำงาน ที่กำหนดไว้ในตัว CPU อีกลักษณะหนึ่งจะเป็นระบบที่ไม่มีการตอบสนองกับสภาวะรอบด้าน แต่ใช้ในการประมวลผลทางข้อมูล หรือการสื่อสาร เพื่อที่จะรองรับการประยุกต์ใช้งานแบบ Real-time เช่น การบีบอัดสัญญาณการเข้ารหัสสัญญาณ ซึ่งจะต้องมีฮาร์ดแวร์มาช่วยในการคำนวณให้สามารถทำงานได้ทันตามที่ต้องการ

- Reconfigurable computing system: ระบบจะประกอบด้วย CPU และฮาร์ดแวร์ที่สามารถโปรแกรมได้นั้นคือ FPGA รวมทั้งหน่วยความจำ ซึ่งสามารถใช้ร่วมกับคอมพิวเตอร์ หรือมี CPU เป็นของตัวเอง สิ่งที่กำหนดความแตกต่างกับ Embedded system ก็คือ ระบบนี้สามารถทำการปรับเปลี่ยนวงจรฮาร์ดแวร์ของตัวเองได้ ตามลักษณะของการประยุกต์ใช้งานที่ต่างกัน ซึ่งการปรับเปลี่ยนนี้สามารถทำได้ก่อนการเริ่มทำงานของระบบ

(Compile-time configuration) หรือทำในขณะที่ระบบมีการทำงาน (Run-time reconfiguration) ดังนั้นจะเห็นได้ว่า ระบบนี้มีความยืดหยุ่นสูงมาก มีการใช้ทรัพยากรฮาร์ดแวร์อย่างคุ้มค่าโดยสามารถนำใช้ใหม่ (Reuse) ได้โดยการโปรแกรมในอุปกรณ์ฮาร์ดแวร์ตัวเดิม ปัญหาก็คือ การออกแบบก็จะต้องมีความซับซ้อนขึ้นเป็นทวีคูณ

2.6 ขั้นตอนการออกแบบ FPGA

2.6.1 ขั้นตอนที่ 1 การสร้างข้อกำหนดของการออกแบบ (Design Specification)

เป็นขั้นตอนการสร้างข้อกำหนดต่างๆ ของวงจร เช่นวงจรงานที่ความถี่เท่าไร ฟังก์ชันมีอะไรบ้าง ซึ่งเป็นรายละเอียดของวงจรที่ต้องการออกแบบและเขียนฟังก์ชันการทำงานของวงจรตามที่ผู้ออกแบบกำหนดด้วยภาษา VHDL ในระดับ RTL

2.6.2 ขั้นตอนที่ 2 จำลองการทำงานโมเดลวงจรระดับ RTL (RTL Simulation)

เป็นขั้นตอนตรวจสอบการทำงานของโมเดลวงจรในระดับ RTL หรือฟังก์ชันโดยการจำลองการทำงาน จะถูกทดสอบด้วย Testbench ซึ่งการจำลองการทำงานในขั้นตอนนี้ จะจำลองการทำงานเพียงฟังก์ชัน ไม่คำนึงถึงค่าดีเลย์ (Delay) ของวงจรแต่อย่างใด

2.6.3 ขั้นตอนที่ 3 สังเคราะห์และออปติไมซ์วงจร (Synthesis & Optimization)

เป็นขั้นตอนการสร้างแผนภาพวงจร (Schematic) จากโมเดลวงจรระดับ RTL ให้อยู่ในรูปแบบของลอจิกเกต โดยอาศัยซอฟต์แวร์ช่วยในการสังเคราะห์วงจร โดยในขั้นตอนนี้จะต้องมีการเลือกใช้

เทคโนโลยี FPGA ที่ผู้ออกแบบต้องการเลือกใช้ ซึ่งบริษัทผู้ผลิต FPGA จะมีเทคโนโลยีไลบรารี (Technology library) เตรียมไว้ให้ผู้ออกแบบไว้ในซอฟต์แวร์ที่ใช้พัฒนาชิป FPGA ของแต่ละบริษัทผู้ผลิตไว้เรียบร้อยแล้ว โดยเมื่อทำการสังเคราะห์ได้ผังวงจร จากนั้นซอฟต์แวร์จะทำการอปติไมซ์วงจรตามข้อกำหนดหรือเงื่อนไขของการสังเคราะห์ ที่ผู้ออกแบบกำหนดขึ้นตามขั้นตอนแรกของการออกแบบ ซึ่งผลลัพธ์ที่ได้จากการสังเคราะห์จะอยู่ในรูปแบบไฟล์ VHDL แบบโครงสร้างลอจิก (Structured VHDL) และไฟล์เน็ตลิสต์มาตรฐาน (Netlist) ประเภท EDIF (Electronic Design Interchange Format) ที่จะนำไปใช้ในขั้นตอน Place & route ต่อไป

2.6.4 ขั้นตอนที่ 4 การจำลองการทำงานของวงจรระดับลอจิกเกต (Gate level Simulation)

เป็นขั้นตอนที่ผู้ออกแบบจะต้องทดสอบไฟล์เน็ตลิสต์ที่เป็นโมเดลของวงจรระดับ

ลอจิก โดยใช้ Testbench ตัวเดิมที่ใช้จำลองการทำงานระดับ RTL มาแล้วซึ่งในการจำลองการทำงานในระดับนี้ จะมีเรื่องของเกตดีเลย์ (Gate delay) เข้ามาเกี่ยวข้องในผลการจำลองการทำงาน ซึ่งจะแตกต่างจากการจำลองการทำงานในระดับ RTL เนื่องจากโมเดลของวงจรระดับเกตนี้ จะมีข้อมูลเรื่องดีเลย์ของเกตภายในเทคโนโลยีที่ผู้ออกแบบเลือกมาใช้ ดังนั้นผู้ออกแบบจะต้องทำการจำลองการทำงานเพื่อตรวจสอบใหม่อีกครั้งหนึ่ง ว่ายังถูกต้องตามข้อกำหนดของวงจรหรือไม่ ถ้าไม่ตรงตามข้อกำหนดจะต้องกลับไปขั้นตอนที่ 3 เพื่อทำการสังเคราะห์และอปติไมซ์วงจรใหม่ เพื่อให้ผลการจำลองการทำงานถูกต้องตาม Design specification

2.6.5 ขั้นตอนที่ 5 การวางและเชื่อมต่อเซลล์ภายในของ FPGA (Place & Route)

เมื่อตรวจสอบการทำงานในระดับลอจิกเกตเป็นที่เรียบร้อยแล้ว เราจะนำไฟล์เน็ตลิสต์ที่อยู่ในรูปแบบ EDIF มาทำการแปลงสู่เทคโนโลยีเซลล์ภายในของ FPGA และทำการเชื่อมต่อเซลล์ภายในเข้าด้วยกัน ตามรูปแบบการเชื่อมต่ออุปกรณ์ต่างๆ ภายในเน็ตลิสต์ โดยขั้นตอนนี้ จะมีการเรียกใช้เทคโนโลยีเซลล์ของ FPGA เนื่องจากเทคโนโลยีเซลล์ของ FPGA อาจมีมาโครเซลล์ (Macro cells) สำหรับสร้างฟังก์ชันต่างๆ ให้กับผู้ออกแบบใช้งานเพราะมาโครเซลล์ต่างๆ พวกนี้ถูกทางผู้ผลิตออกแบบไว้สำหรับชิป FPGA แต่ละตัว โดยผลลัพธ์จากขั้นตอนนี้ จะอยู่ในรูปของไฟล์ VHDL โครงสร้างของเซลล์ภายในของ FPGA และไฟล์ประเภท SDF (Standard Delay Format) ซึ่งเป็นไฟล์รูปแบบมาตรฐานที่มีข้อมูลเกี่ยวกับค่าดีเลย์ของเส้นทางการเชื่อมต่อภายใน (Routing delay) เซลล์ FPGA และไฟล์ที่ใช้สำหรับไปโปรแกรมชิป FPGA ซึ่งส่วนใหญ่จะอยู่ในรูปแบบของ Configuration file

2.6.6 ขั้นตอนที่ 6 การจำลองการทำงานวงจรระดับฐานเวลาจริง (Timing Simulation)

เป็นขั้นตอนสุดท้ายของการตรวจสอบความถูกต้อง ก่อนจะนำวงจรที่ออกแบบไปโปรแกรมหรือบางครั้งเรียกว่า Download ลงสู่ชิปจริงต่อไป โดยขั้นตอนการจำลองการทำงานในระดับไทม์มิง (Timing model) โดยผลลัพธ์จากการจำลองการทำงานที่ได้จะมีความใกล้เคียงกับไทม์มิงการทำงานจริงบนชิป FPGA เนื่องจากในการจำลองการทำงานในขั้นตอนนี้มีข้อมูลเกี่ยวกับดีเลย์ของเซลล์ภายใน FPGA และดีเลย์ของการเชื่อมต่อเซลล์เข้ามาเกี่ยวข้อง ทำให้ผลการจำลองการทำงานในระดับนี้ใกล้เคียงกับไทม์มิงการทำงานจริงบนฮาร์ดแวร์

2.6.7 ขั้นตอนที่ 7 โปรแกรมลงสู่ชิปจริง (Download to device)

เป็นขั้นตอนสุดท้ายสำหรับการออกแบบวงจรเพื่อใช้งานชิป FPGA คือขั้นตอนโปรแกรม Configuration file ลงสู่ชิป FPGA บนบอร์ดเพื่อทดสอบการทำงานจริงต่อไป